

Aula de hoje: botão de compra novo e upgrade dos blocos

08/10/2026 · Instrutor Pedro Leonardo

O que vamos fazer hoje

O dropper já funciona do começo ao fim. Hoje vamos melhorar duas coisas: o jeito de vender os itens da base e o valor dos blocos no caminho da esteira.

- **Botão de compra novo:** esconde e mostra os itens sozinho. Funciona com Models, Unions e itens da Toolbox, e permite montar a base em andares.
- **Upgrade dos blocos:** um portal na esteira que aumenta o valor de cada bloco (box) que passa por ele e coloca um brilho nele.

No fim da aula, o caminho do dinheiro vai ficar assim:

Gerador → Bloco → Esteira → Upgrade → Destino final → Armazém → Botão de coleta → Carteira

Lembrando a regra de sempre: as configurações (preço, multiplicador, cor) ficam em variáveis no topo do script. Quer mudar o jogo? Mude só essas linhas.

Etapa 1: preparar a base para o botão novo

Por quê: até agora, cada item à venda precisava ser deixado invisível e sem colisão à mão no Studio. Isso dá trabalho e não funciona com itens da Toolbox, que têm peças invisíveis de propósito, luzes, sons e scripts próprios. O botão novo resolve isso: quando o jogo começa, ele anota como cada coisa era, esconde tudo e, na compra, devolve exatamente como estava.

Como fazer:

- Deixe visíveis de novo todos os itens que você escondeu à mão (gerador, esteira, destino, paredes...): **Transparency** em 0 e **CanCollide** marcado. Monte a base do jeito que ela vai ficar depois de comprada.
- Confira se a peça de todo botão se chama exatamente Botao. O script usa esse nome para reconhecer os outros botões.
- Os itens podem ficar soltos na pasta, dentro de Models ou de subpastas. Tanto faz.

Etapa 2: o script do botão

Como fazer: troque o script de **todos** os botões por este. De um botão para outro, a única linha que muda é a do preço.

```
-- =====
-- SCRIPT: BOTÃO DE COMPRA
-- Esconde tudo o que está na pasta do botão quando o jogo começa
-- e mostra de volta, do jeito original, quando o jogador compra.
-- Funciona com Parts, Unions, MeshParts, Models e itens da Toolbox.
-- Botões podem ficar dentro da pasta de outro botão: eles só
-- aparecem depois que o botão de fora for comprado.
-- Local: dentro da peça Botao (ex: Workspace.Botao1.Botao)
-- IMPORTANTE: toda peça de botão precisa se chamar "Botao"
-- =====
```

```

local ServerScriptService = game:GetService("ServerScriptService")
local Players = game:GetService("Players")
local Economia = require(ServerScriptService.Economia)

-- a própria peça do botão e a pasta onde ele está
local botao = script.Parent
local pasta = botao.Parent

-- =====
-- CONFIGURAÇÕES DO BOTÃO
-- =====
local preco = 10 -- quanto custa liberar os itens desta pasta
local ancorarAoComprar = true -- true: tudo fica preso no lugar depois da compra (bom para
decoração)
-- false: cada peça volta exatamente como era (bom para NPCs, portas, veículos)

-- =====
-- VARIÁVEIS DE CONTROLE
-- =====
local jaComprado = false -- impede comprar 2x ao mesmo tempo
local liberado = true -- false enquanto o botão de fora ainda não foi comprado
local originalItens = {} -- como cada item da pasta era antes de ser escondido
local originalBotao = {} -- como o próprio botão era antes de ser escondido

-- tipos de objeto que têm a propriedade "Enabled" (ligado/desligado)
local tiposComEnabled = {
    "Script", "ParticleEmitter", "Light", "Fire", "Smoke", "Sparkles",
    "Beam", "Trail", "SurfaceGui", "BillboardGui", "ProximityPrompt",
}

-- =====
-- FUNÇÕES DE AJUDA
-- =====

-- confere se o objeto é de algum tipo da lista tiposComEnabled
local function temEnabled(objeto)
    for _, tipo in ipairs(tiposComEnabled) do
        if objeto:IsA(tipo) then
            return true
        end
    end
    return false
end

-- confere se o objeto é o botão ou algo dentro dele
local function ehDoBotao(objeto)
    return objeto == botao or objeto:IsDescendantOf(botao)
end

-- confere se o objeto está na pasta de OUTRO botão (um botão de dentro)
-- sobe pelas pastas do objeto até chegar na nossa pasta;
-- se no caminho achar uma pasta com outro "Botao", o objeto é dele

```

```

local function ehDeOutroBotao(objeto)
    local atual = objeto.Parent
    while atual and atual ~= pasta do
        local outro = atual:FindFirstChild("Botao")
        if outro and outro ~= botao and outro:IsA("BasePart") then
            return true
        end
        atual = atual.Parent
    end
    return false
end

-- procura um botão nas pastas ACIMA da nossa (o botão "pai")
-- se achar, este botão só aparece depois que aquele for comprado
local function acharBotaoPai()
    local atual = pasta.Parent
    while atual and atual ~= game do
        local outro = atual:FindFirstChild("Botao")
        if outro and outro:IsA("BasePart") then
            return outro
        end
        atual = atual.Parent
    end
    return nil
end

-- anota como o objeto é agora (na tabela escolhida) e depois esconde/desliga ele
local function esconder(objeto, tabela)
    if objeto:IsA("BasePart") then
        -- qualquer peça física: Part, Union, MeshPart, Wedge...
        tabela[objeto] = {
            Transparency = objeto.Transparency,
            CanCollide = objeto.CanCollide,
            CanTouch = objeto.CanTouch,
            CanQuery = objeto.CanQuery,
            Anchored = objeto.Anchored,
        }
        objeto.Transparency = 1
        objeto.CanCollide = false
        objeto.CanTouch = false
        objeto.CanQuery = false
        objeto.Anchored = true -- escondida e sem colisão, ela cairia do mapa

    elseif objeto:IsA("Decal") then
        -- imagens coladas nas peças (Texture também conta como Decal)
        tabela[objeto] = { Transparency = objeto.Transparency }
        objeto.Transparency = 1

    elseif objeto:IsA("Sound") then
        tabela[objeto] = { Playing = objeto.Playing }
        objeto:Stop()

    elseif temEnabled(objeto) then

```

```

    tabela[objeto] = { Enabled = objeto.Enabled }
    objeto.Enabled = false
end
end

-- devolve as propriedades que foram anotadas
local function restaurar(objeto, propriedades)
    for nome, valor in pairs(propriedades) do
        objeto[nome] = valor
    end
    if ancorarAoComprar and objeto:IsA("BasePart") then
        objeto.Anchored = true
    end
end

-- mostra de volta tudo o que está numa tabela:
-- primeiro peças e efeitos, depois os scripts
local function mostrarTudo(tabela)
    local scripts = {}
    for objeto, propriedades in pairs(tabela) do
        if objeto.Parent then -- pula objetos destruídos no meio do jogo
            if objeto:IsA("Script") then
                table.insert(scripts, objeto)
            else
                restaurar(objeto, propriedades)
            end
        end
    end
    for _, s in ipairs(scripts) do
        restaurar(s, tabela[s])
    end
end

-- esconde o próprio botão (e o que tiver dentro dele, como textos de preço)
-- o script que está rodando agora nunca é desligado
local function esconderBotao()
    esconder(botao, originalBotao)
    for _, objeto in ipairs(botao:GetDescendants()) do
        if objeto ~= script then
            esconder(objeto, originalBotao)
        end
    end
end

-- =====
-- QUANDO O JOGO COMEÇA
-- =====

-- 1) esconde os itens desta pasta
-- (pula o próprio botão e tudo o que pertence a botões de dentro)
for _, objeto in ipairs(pasta:GetDescendants()) do
    if not ehDoBotao(objeto) and not ehDeOutroBotao(objeto) then
        esconder(objeto, originalItens)
    end
end

```

```

    end
end

-- 2) se existe um botão acima deste, este botão começa escondido
-- e só aparece quando aquele for comprado
local botaoPai = acharBotaoPai()

if botaoPai then
    liberado = false
    esconderBotao()

    local function aparecer()
        if liberado then return end
        liberado = true
        mostrarTudo(originalBotao)
    end

    -- o botão de cima ganha a marca "Comprado" quando é comprado
    botaoPai:GetAttributeChangedSignal("Comprado"):Connect(function()
        if botaoPai:GetAttribute("Comprado") then
            aparecer()
        end
    end)

    -- se o botão de cima já tiver sido comprado, aparece na hora
    if botaoPai:GetAttribute("Comprado") then
        aparecer()
    end
end

=====
-- EVENTO: dispara toda vez que algo toca no botão
=====

botao.Touched:Connect(function(hit)
    if jaComprado or not liberado then return end -- já comprado ou ainda bloqueado

    -- descobre se quem tocou é um jogador (e não um objeto qualquer)
    local jogador = Players:GetPlayerFromCharacter(hit.Parent)
    if not jogador then return end

    -- confere quanto dinheiro o jogador tem na carteira
    local carteira = Economia.carteiras[jogador.UserId] or 0

    if carteira >= preco then
        jaComprado = true -- trava a compra na hora

        -- desconta o preço e libera os itens desta pasta
        Economia.carteiras[jogador.UserId] = carteira - preco
        mostrarTudo(originalItens)

        -- o próprio botão some (e o que tiver dentro dele, como textos de preço)
        esconderBotao()
    end
end)

```

```
-- marca este botão como comprado: os botões de dentro vão aparecer
botao:SetAttribute("Comprado", true)

print(jogador.Name .. " comprou por " .. preco)
else
    print(jogador.Name .. " não tem dinheiro suficiente")
end
end)
end)
```

Etapa 3: entendendo o script por partes

O script é grande, mas é feito de pedaços pequenos. Cada um tem um trabalho só.

Configurações

- `preco` é quanto custa a compra.
- `ancorarAoComprar` decide se tudo fica preso no lugar depois da compra. Deixe `true` para decoração e paredes. Use `false` só para coisas que precisam se mexer, como NPCs, portas ou veículos.

Variáveis de controle

- `jaComprado` impede que o botão seja comprado duas vezes.
- `liberado` fica `false` enquanto o botão está esperando o botão de baixo ser comprado.
- `originalItens` e `originalBotao` são tabelas, como cadernos de anotação: guardam como cada objeto era antes de ser escondido.
- `tiposComEnabled` é a lista de objetos que se ligam e desligam: luzes, partículas, fogo, textos, scripts.

Funções de ajuda

- `temEnabled` responde se um objeto é de um dos tipos da lista.
- `ehDoBotao` responde se um objeto é o próprio botão ou está dentro dele.
- `ehDeOutroBotao` responde se um objeto pertence a um botão de dentro, como o do segundo andar. Esses objetos não são mexidos por este botão.
- `acharBotaoPai` sobe pelas pastas procurando outro `Botao` acima. Se achar, este botão é de um andar de cima.

Esconder e mostrar

- `esconder` anota as propriedades do objeto no caderno e depois esconde ou desliga. Peças ficam invisíveis, sem colisão e ancoradas. Imagens ficam transparentes. Sons param. Luzes, efeitos e scripts são desligados.
- `restaurar` lê o caderno e devolve cada propriedade ao valor anotado.
- `mostrarTudo` restaura tudo de uma tabela. Ele liga os scripts por último, para que, quando um script começar, as peças dele já estejam visíveis.
- `esconderBotao` esconde o botão e o que estiver dentro dele, como um texto de preço, menos o próprio script que está rodando.

Quando o jogo começa

- `GetDescendants` pega tudo dentro da pasta, em qualquer nível, e o script esconde cada coisa.
- Se existe um botão acima, este botão também se esconde e fica observando a marca `Comprado` do botão de baixo com `GetAttributeChangedSignal`. Quando a marca aparece, ele aparece também.

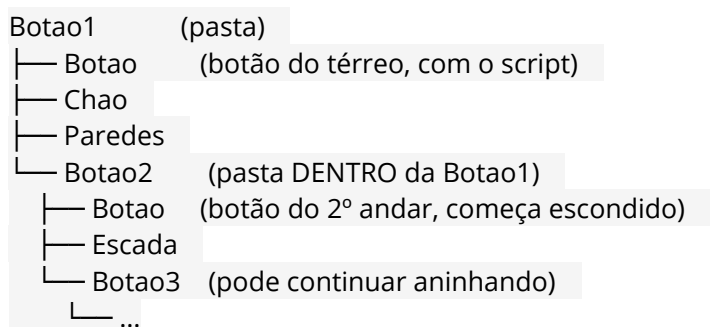
A compra

- Igual ao botão antigo: confere se quem tocou é um jogador, se ele tem dinheiro e desconta o preço.
- Depois mostra os itens, esconde o botão e marca o botão com `SetAttribute("Comprado", true)`. Essa marca é o aviso para os botões do andar de cima.

Etapa 4: montar a base em andares

Por quê: se todos os botões aparecem desde o começo, os botões do segundo andar ficam flutuando no ar antes de existir chão. Com pastas uma dentro da outra, cada botão só aparece quando o de baixo é comprado.

Como fazer: arraste a pasta do botão de cima para dentro da pasta do botão de baixo:



- Comprar o `Botao1` mostra o chão e as paredes, e faz o botão do `Botao2` aparecer. O que está dentro da `Botao2` continua escondido até ela ser comprada.
- Para dois botões aparecerem depois da mesma compra (por exemplo, "segundo andar" e "garagem"), coloque as duas pastas dentro da mesma pasta de fora.

Etapa 5: a peça do upgrade

Por quê: em um tycoon, o jogador precisa de motivos para continuar comprando. O upgrade é uma máquina que deixa cada bloco mais valioso, sem precisar de um gerador novo.

Como fazer:

- Insira uma **Part** chamada `Upgrade` em cima da esteira, no meio do caminho.
- Deixe a peça larga e alta o bastante para o bloco passar por dentro, por exemplo `Size 5, 4, 1`.
- Use o material **Neon** e `Transparency` entre `0.5` e `0.7`, para parecer um portal.
- Para vender o upgrade num botão, coloque a peça dentro da pasta do botão, como qualquer outro item.

Etapa 6: script do upgrade

Como fazer: dentro de Upgrade, insira um **Script** chamado Upgrade e cole o código:

```
-- =====
-- SCRIPT: UPGRADE
-- Portal colocado no caminho da esteira.
-- Quando um bloco passa por ele, o valor do bloco aumenta
-- e o bloco ganha um efeito de brilho (ou muda a cor do brilho que já tem).
-- O jogador consegue atravessar o portal normalmente.
-- Local: dentro da peça Upgrade
-- =====

local upgrade = script.Parent

-- =====
-- CONFIGURAÇÕES DO UPGRADE
-- =====

local multiplicador = 2 -- o valor do bloco é multiplicado por isso
local bonus = 0 -- e depois somado com isso
local corDoEfeito = Color3.fromRGB(0, 255, 255) -- cor do brilho que o bloco ganha
local nomeDoEfeito = "EfeitoUpgrade" -- nome do efeito (todos os upgrades usam o mesmo)

-- o portal fica parado, não bloqueia ninguém, mas percebe quem passa por dentro
upgrade.Anchored = true
upgrade.CanCollide = false -- jogador e blocos atravessam
upgrade.CanTouch = true -- mesmo atravessando, o Touched continua funcionando

-- lista dos blocos que já passaram por ESTE upgrade
-- (para o mesmo bloco não ser melhorado várias vezes)
local jaMelhorados = {}

-- =====
-- FUNÇÃO: coloca o efeito no bloco, ou muda a cor se ele já tiver
-- =====

local function aplicarEfeito(bloco)
    -- procura se o bloco já tem um efeito de upgrade
    local efeito = bloco:FindFirstChild(nomeDoEfeito)

    -- se não tem, cria um novo
    if not efeito then
        efeito = Instance.new("ParticleEmitter")
        efeito.Name = nomeDoEfeito
        efeito.Rate = 15 -- partículas por segundo
        efeito.Lifetime = NumberRange.new(0.4, 0.8) -- quanto tempo cada partícula dura
        efeito.Speed = NumberRange.new(1, 2) -- velocidade das partículas
        efeito.Size = NumberSequence.new(0.25) -- tamanho das partículas
        efeito.LightEmission = 1 -- deixa as partículas brilhando
        efeito.Parent = bloco
    end

    -- tendo ou não tendo antes, o efeito fica com a cor DESTES upgrade
    efeito.Color = ColorSequence.new(corDoEfeito)
end
```

```

-- =====
-- EVENTO: dispara toda vez que algo passa por dentro do upgrade
-- =====
upgrade.Touched:Connect(function(hit)
    -- só blocos têm a etiqueta "Valor"; jogador e outras peças são ignorados
    local valor = hit:GetAttribute("Valor")
    if not valor then return end

    -- se este bloco já passou por aqui, ignora
    if jaMelhorados[hit] then return end
    jaMelhorados[hit] = true

    -- quando o bloco for destruído, tira ele da lista (para a lista não crescer pra sempre)
    hit.Destroying:Once(function()
        jaMelhorados[hit] = nil
    end)

    -- calcula o novo valor e arredonda (para não virar 7.5, por exemplo)
    local novoValor = math.floor(valor * multiplicador + bonus + 0.5)
    hit:SetAttribute("Valor", novoValor)

    aplicarEfeito(hit)

    print("Upgrade! Bloco foi de " .. valor .. " para " .. novoValor)
end)

```

Entendendo o código:

- `multiplicador` e `bonus` decidem o novo valor. Com 2 e 0, o valor dobra. Com 1 e 10, soma 10. Dá para usar os dois juntos.
- `CanCollide = false` deixa o jogador e os blocos atravessarem. `CanTouch = true` faz o `Touched` continuar funcionando mesmo assim.
- `GetAttribute("Valor")` é a mesma regra do destino final: só os blocos têm essa etiqueta, então só eles são afetados.
- `jaMelhorados` é a lista dos blocos que já passaram por este upgrade. Sem ela, o `Touched` dispararia várias vezes enquanto o bloco atravessa, e o valor seria multiplicado muitas vezes.
- `Destroying` avisa quando o bloco é destruído, e o bloco sai da lista. Assim a lista não cresce para sempre.
- `math.floor(... + 0.5)` arredonda o valor, para não aparecer dinheiro quebrado, como 7.5.
- `aplicarEfeito` procura o efeito `EfeitoUpgrade` no bloco. Se não existe, cria as partículas. Se já existe, só troca a cor. Assim o jogador vê, pela cor, qual foi o último upgrade por onde o bloco passou.

Dois upgrades na mesma esteira somam efeito: um bloco de 5 vira 10 no primeiro (×2) e 20 no segundo (×2).

Testar e problemas comuns

Dê **Play** e confira:

- Ao iniciar, todos os itens à venda somem e só os botões do térreo aparecem
- Ao comprar, os itens aparecem do jeito que estavam no Studio
- O botão do segundo andar só aparece depois da compra do térreo
- Ao passar pelo upgrade, o bloco ganha brilho e o valor no Output aumenta
- O jogador atravessa o upgrade sem bater

Problema	Causa provável	Solução
O item comprado aparece invisível ou sem colisão	O item ainda estava escondido à mão no Studio, do jeito antigo	Deixe Transparency em 0 e marque CanCollide no Studio
O gerador não solta blocos depois da compra	O gerador continua invisível no Studio, e o script dele confere a transparência	Deixe o gerador visível no Studio (Transparency 0)
O botão do 2º andar aparece desde o começo	A pasta dele não está dentro da pasta do botão de baixo	Arraste a pasta Botao2 para dentro da Botao1
Comprar o térreo mostra o 2º andar inteiro	A peça do botão de dentro não se chama Botao	Renomeie a peça para Botao, exatamente assim
Um modelo da Toolbox desmonta ou cai depois da compra	ancorarAoComprar está em false	Mude para true no botão desse modelo
O bloco passa pelo upgrade e nada muda	O bloco passa por fora do portal, ou o upgrade ainda não foi comprado	Aumente o tamanho do portal e confira se ele foi comprado

Próximas aulas

- **Dono da base:** um botão para pegar a base, e só o dono pode comprar nela.
- **Itens da Toolbox à venda:** poderes e armas que o jogador pode comprar na base.

Tarefa em aula

- **Faça toda a implementação do arquivo**
- **Monte a base com pelo menos 2 andares**
- **Use pelo menos 2 droppers diferentes**
- **Crie pelo menos 2 tipos de bloco diferentes**
- **Crie pelo menos 2 upgrades diferentes**
- **Tenha 10 botões de compra no total**